



PUCE
Pontificia Universidad
Católica del Ecuador

PUCE VIRTUAL

Facultad de Ingeniería
Ingeniería en Ciberseguridad

DESARROLLO DE PLATAFORMAS

T1S5 · Unidad 5

Justificación técnica

**API REST de incidentes con persistencia
en MySQL, Prisma ORM y front-end**

Docker · Prisma 7 · migraciones · CRUD persistente

Autor

Alejandro David Carriel Zambrano

1. Para qué es esta entrega

Esta justificación explica las decisiones al cambiar el modelo en memoria de la Unidad 4 por una base relacional de verdad. La rúbrica pedía persistencia real, variables de entorno, modelo y migraciones, CRUD, filtros por estado o prioridad, validaciones y manejo central de excepciones, con pruebas de casos correctos y de error. En corto: que si apago Node, los incidentes sigan ahí.

2. De la RAM a un motor relacional

En Semana 4 el almacén era un arreglo dentro del proceso de Node. Al detener el servidor los casos nuevos se esfumaban. Servía para aislar Express y el contrato JSON, pero en un SOC perder la bitácora no es aceptable. Elegí MySQL 8, el motor del laboratorio, porque el incidente tiene campos fijos y filtros exactos. NoSQL acá no aporta: no hay documentos anidados. SQL te da integridad y un esquema versionado.

3. Docker como entorno, no como adorno

MySQL corre en el contenedor `mysql-incidentes` (puerto 3306, base `incidentes`). Quien clone el proyecto no tiene que instalar el motor en Windows. Node sigue en el host; Docker solo aísla la base. Reiniciar Node no borra filas. Destruir el volumen del contenedor sí. Esa distinción es, para mí, la prueba de persistencia.

4. Prisma 7, adapter y el secreto fuera del código

Usé Prisma 7.10 con `@prisma/adapter-mariadb`, como en clase. El adapter habla el protocolo de MySQL. La URL vive en `DATABASE_URL` (`.env` + `prisma7.config.ts` y `dotenv`). El `.env` no se versiona; se entrega `.env.example`.

El modelo `Incidente` tiene `id` autonumérico, `codigo` único para el front, `severidad`, `prioridad`, `estado` y el resto del dominio. `Prioridad` y `estado` cubren el filtro de la rúbrica. La migración `init` dejó SQL en `prisma/migrations` y Studio mostró la tabla vacía antes de la semilla.

5. Capas: el ORM no se comió la arquitectura

La demo del docente cabía en un `server.js`. Acá lo repartí: `prisma.js` crea el cliente; el modelo reemplaza el array por `findMany`, `create` y `update` y proyecta `id = codigo`; el servicio valida antes de escribir; el controlador mantiene el CRUD y lee `prioridad` y `estado` en la query; el middleware sigue respondiendo `{ ok, mensaje, detalles }`. Así un 400 nace de la regla de negocio, no de un error opaco del motor.

6. Pruebas: acierto, filtro, error y reinicio

Tres casos de semilla en Studio llenaron el tablero. El alta desde la web escribió la fila 4. Tras `Ctrl+C` y `npm start` los cuatro siguieron visibles. Thunder, sobre `/api` y no sobre el HTML, documentó el contrato: `GET ?prioridad=alta` (200, un ítem), `GET` de un código que no existe (404), `POST` inválido (400 con detalles) y `POST` válido (201). El filtro es un `where` de Prisma, no un `filter` en RAM.

7. Decisiones conscientes y límites

Conservé el `DELETE` lógico de Semana 4 (pasar a `Cerrado`) porque un SOC no borra evidencia; el alta física sí es `INSERT`. No subí de Prisma 7 a 8: el laboratorio usó 7.10 y un salto mayor no aporta a la rúbrica. Todavía no hay usuarios ni JWT; la «seguridad» de esta unidad es no filtrar la clave y validar entrada.

El front aún filtra sobre todo por severidad; prioridad y estado los demostré en Thunder, que es el canal que el docente usó para el contrato. Queda pendiente un seed automatizado y tests de integración; para T1S5 basta el ciclo visible crear–reiniciar–seguir viendo el dato.

8. Cierre

SecureGuard dejó de ser una API de demostración en RAM. Docker sostiene MySQL, Prisma versiona el esquema, `.env` esconde la URL, el servicio valida, el modelo persiste y el front consume el mismo contrato de Semana 4. La prueba de reinicio y las respuestas 201, 400 y 404 en Thunder Client son la evidencia de que la Unidad 5 quedó aplicada sobre la plataforma previa, no como un script aparte.

Anexo. Evidencias

Las figuras siguen el orden del laboratorio: migración, tabla vacía, front vacío, semilla, alta por UI, reinicio y pruebas en consola / Thunder.

```

1  import "dataloader";
2  import { defineConfig } from "prisma/config";
3
4  export default defineConfig({
5    schema: "prisma/schema.prisma",
6    migrations: [
7      {
8        path: "prisma/migrations",
9      },
10   ],
11   datasource: {
12     url: process.env["DATABASE_URL"],
13   },
14 });

```

```

PS C:\Users\carri\Desktop\PUCE\Quinto semestre\Desarrollo de plataformas\UNIFORME DE DESARROLLO\Desarrollo de
plataforma\SEPMVA > npm prisma migrate dev --name init
prisma:prisma@
  ~ 20200908052527_init/
    migration.sql

Your database is now in sync with your schema.

PS C:\Users\carri\Desktop\PUCE\Quinto semestre\Desarrollo de plataformas\UNIFORME DE DESARROLLO\Desarrollo de
plataforma\SEPMVA > npm prisma generate
Loaded Prisma schema from prisma.config.ts.
Prisma schema loaded from prisma/schema.prisma.

Update available 7.10.0 -> 7.10.1
This is a major update - please follow the guide at
https://pris.ly/d/major-version-upgrade
Run the following to update
npm i --save-dev prisma@latest
npm i @prisma/client@latest

✓ Generated Prisma Client (v7.10.0) to .\node_modules\prisma\client in 5ms
Start by importing your Prisma Client (See: https://pris.ly/d/importing-client)

PS C:\Users\carri\Desktop\PUCE\Quinto semestre\Desarrollo de plataformas\UNIFORME DE DESARROLLO\Desarrollo de
plataforma\SEPMVA >

```

Figura 1. Migración init aplicada y cliente Prisma 7.10 generado.

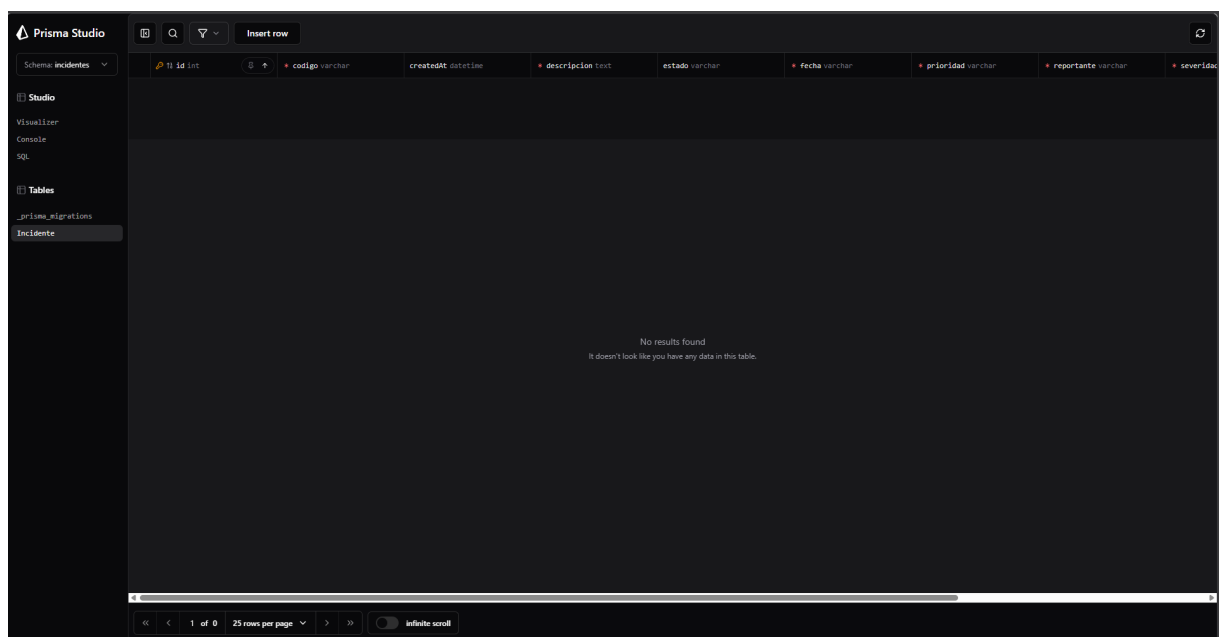


Figura 2. Prisma Studio: tabla Incidente recién creada, sin filas.

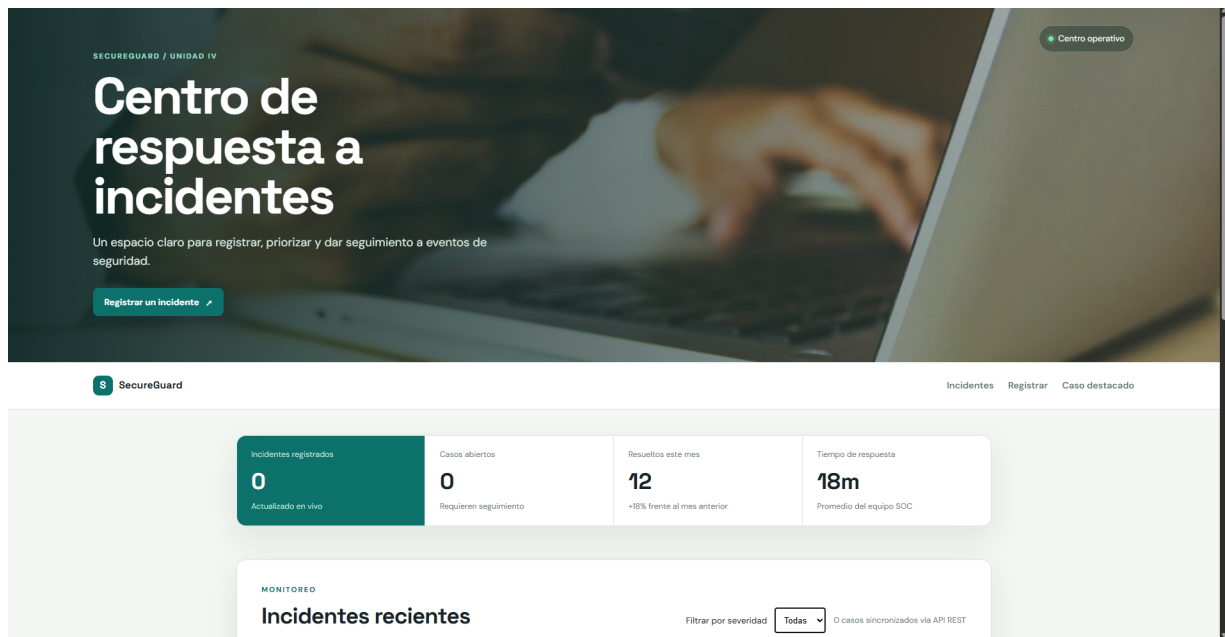


Figura 3. Front-end con 0 incidentes antes de la semilla.

id	codigo	createdAt	description	estado	fecha	prioridad	reportante	severidad	tipo
INC-001		2026-09-30T06:19:31.666Z	Correo masivo solicitand...	En investigación	2026-09-03	alta	so@secureguard.edu	alta	Phishing
INC-002		2026-09-30T09:03:15.879Z	Inicio de sesión atípico e...	Contención en curso	2026-09-02	critica	adm@secureguard.edu	critica	Acceso no autorizado
INC-003		2026-09-30T09:05:45.828Z	Hallazgo de configuración ...	Resuelto	2026-09-01	media	app@secureguard.edu	media	Vulnerabilidad

Figura 4. Semilla INC-001 a INC-003 persistida en MySQL.

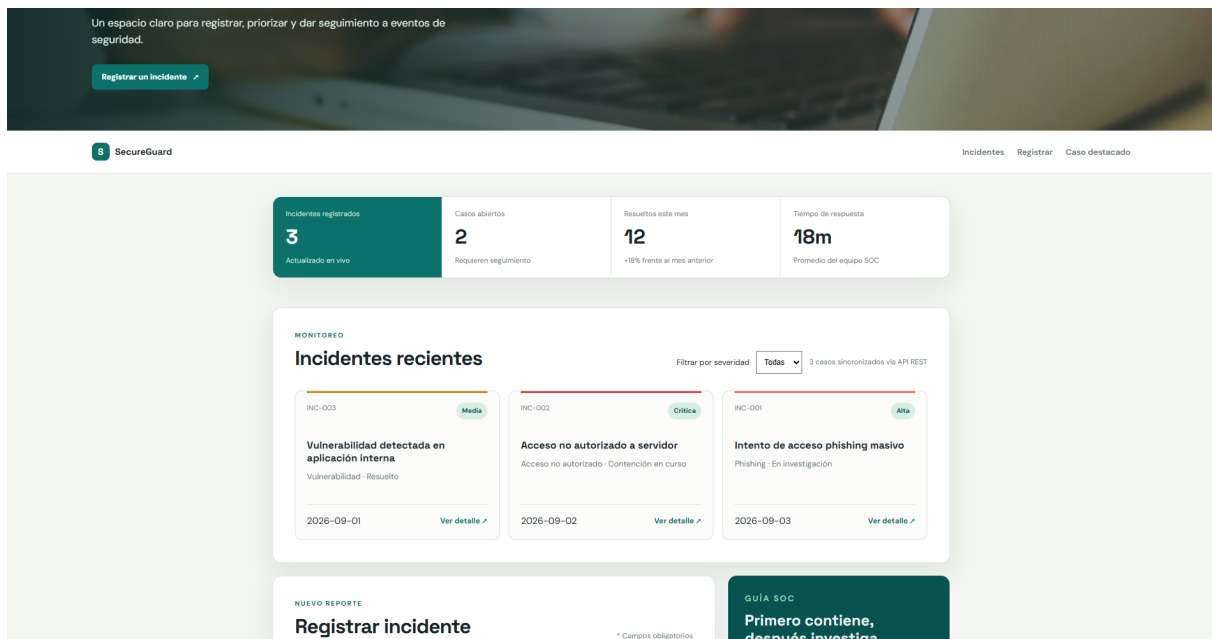


Figura 5. El listado consume la API y muestra los tres casos.

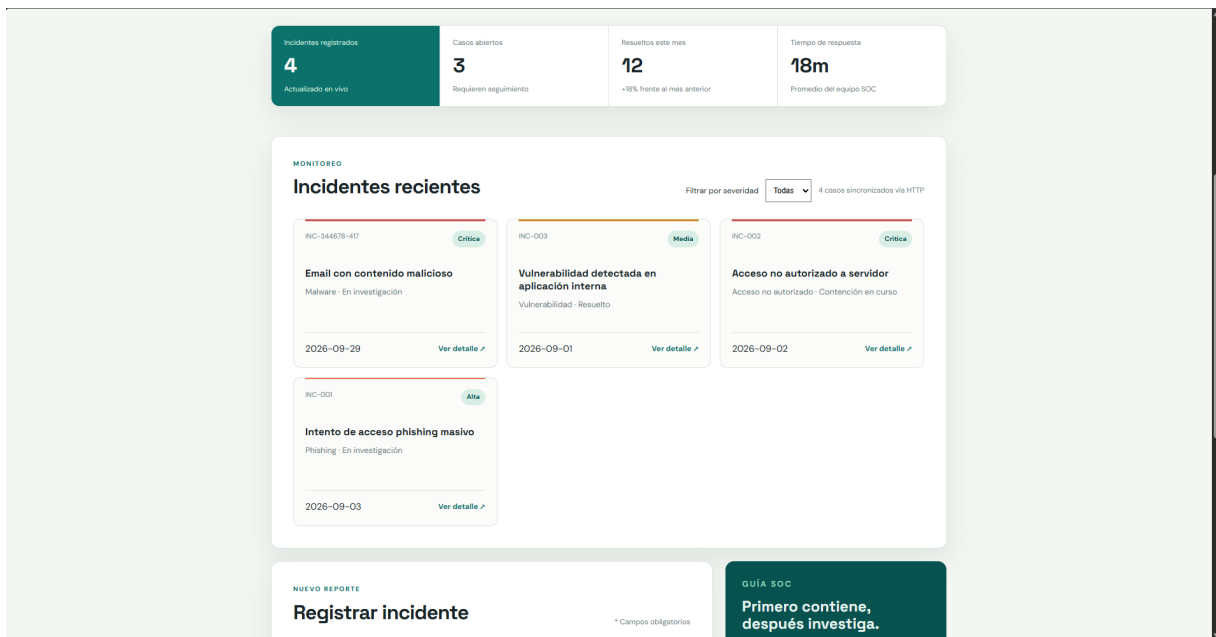
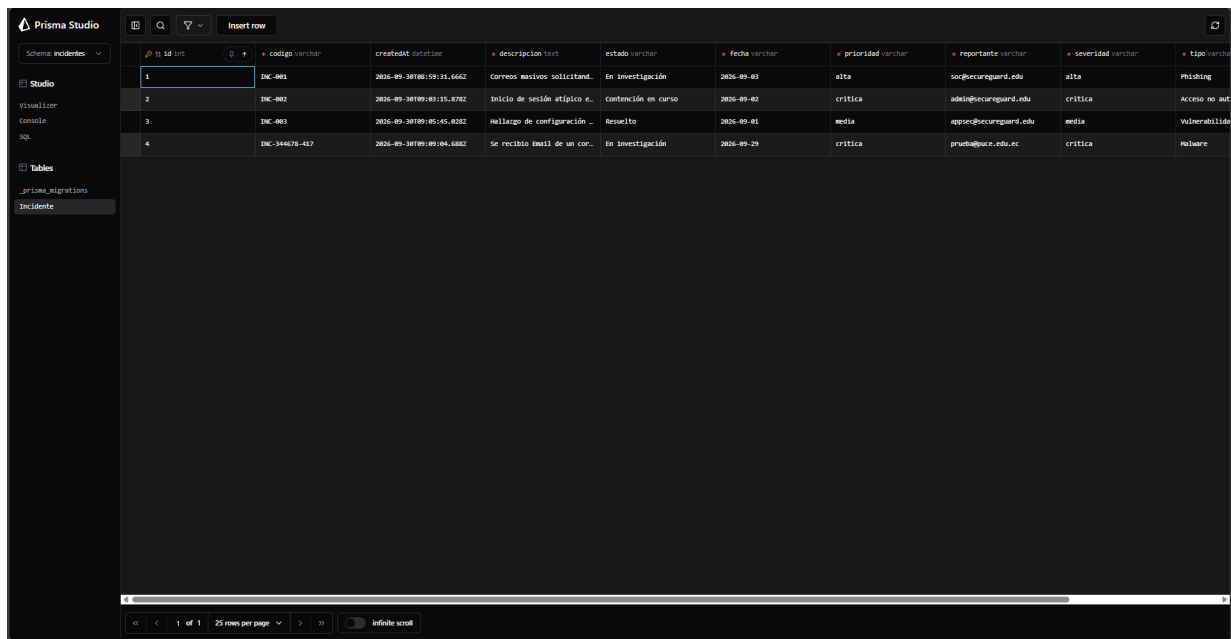
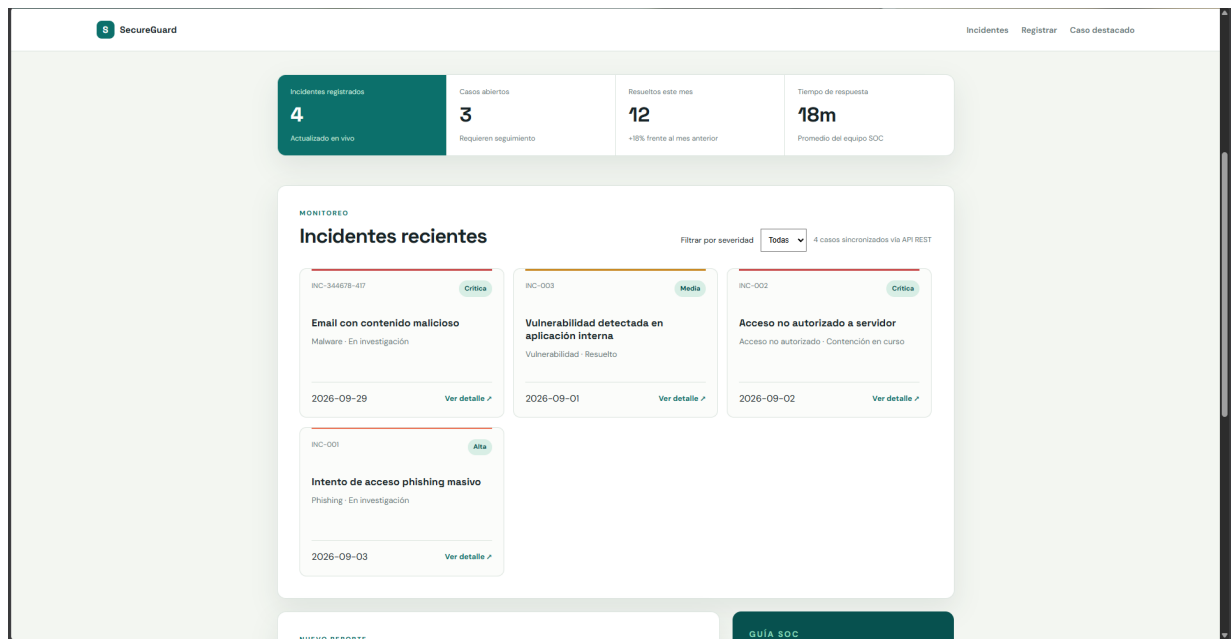


Figura 6. Alta desde el sitio: cuarto caso visible en el tablero.



id	codigo	createdat	descripcion	estado	fecha	prioridad	reportante	severidad	tipo
1	INC-001	2026-09-30T08:59:33.666Z	Correos masivos solicitand.	En investigación	2026-09-01	alta	soc@secureguard.edu	alta	Phishing
2	INC-002	2026-09-30T09:02:15.878Z	Inicio de sesión atípico e.	Contención en curso	2026-09-02	critica	admin@secureguard.edu	critica	Acceso no aut.
3	INC-003	2026-09-30T09:05:45.028Z	Malware de configuración ..	Resuelto	2026-09-01	media	appsec@secureguard.edu	media	Vulnerabilida
4	INC-344678-417	2026-09-30T09:09:04.688Z	Se recibió email de un cor..	En investigación	2026-09-29	critica	prueba@puce.edu.ec	critica	Malware

Figura 7. Studio confirma la fila 4 escrita por el POST del formulario.



SecureGuard

Incidentes Registrados: **4** (Actualizado en vivo)

Casos abiertos: **3** (Requieren seguimiento)

Resueltos este mes: **12** (+18% frente al mes anterior)

Tiempo de respuesta: **18m** (Promedio del equipo SOC)

MONITOREO Incidentes recientes

Filtrar por severidad: **Todos** (4 casos sincronizados via API REST)

- INC-344678-417** (Crítica): Email con contenido malicioso. Malware - En investigación. 2026-09-29. [Ver detalle](#)
- INC-003** (Media): Vulnerabilidad detectada en aplicación interna. Vulnerabilidad - Resuelto. 2026-09-01. [Ver detalle](#)
- INC-002** (Crítica): Acceso no autorizado a servidor. Acceso no autorizado - Contención en curso. 2026-09-02. [Ver detalle](#)
- INC-001** (Alta): Intento de acceso phishing masivo. Phishing - En investigación. 2026-09-03. [Ver detalle](#)

GUÍA SOC

Figura 8. Tras detener y volver a iniciar Node, los cuatro casos siguen.

```
[2026-09-30T03:54:37.147Z] GET /css/estilos.css
[2026-09-30T03:54:37.147Z] GET /js/controlador.js
[2026-09-30T03:54:37.150Z] GET /js/modelo.js
[2026-09-30T03:54:37.151Z] GET /js/vista.js
[2026-09-30T03:54:37.157Z] GET /api/incidentes
[2026-09-30T03:54:41.043Z] GET /api/incidentes?severidad=critica
[2026-09-30T03:54:42.540Z] GET /api/incidentes
[2026-09-30T04:06:00.986Z] GET /index.html
[2026-09-30T04:06:01.007Z] GET /css/estilos.css
[2026-09-30T04:06:01.007Z] GET /js/controlador.js
[2026-09-30T04:06:01.010Z] GET /js/modelo.js
[2026-09-30T04:06:01.010Z] GET /js/vista.js
[2026-09-30T04:06:01.020Z] GET /api/incidentes
[2026-09-30T04:09:04.533Z] POST /api/incidentes
[2026-09-30T04:09:46.548Z] GET /
[2026-09-30T04:09:46.555Z] GET /css/estilos.css
[2026-09-30T04:09:46.557Z] GET /js/controlador.js
[2026-09-30T04:09:46.562Z] GET /js/modelo.js
[2026-09-30T04:09:46.563Z] GET /js/vista.js
[2026-09-30T04:09:46.565Z] GET /api/incidentes
[2026-09-30T04:09:46.574Z] GET /css/estilos.css
PS C:\Users\carri\Desktop\PUCE\Quinto semestre\Desarrollo de plataformas\ENTORNO DE DESARROLLO\Desarrollo de plataforma\SEMANA 5> npm start

> start
> node server.js

? injected env (2) from .env
? injected env (0) from .env
SecureGuard - Unidad 4
Cliente: http://localhost:3000
salud: http://localhost:3000/api/salud
[2026-09-30T04:10:09.081Z] GET /index.html
[2026-09-30T04:10:09.116Z] GET /css/estilos.css
[2026-09-30T04:10:09.117Z] GET /js/controlador.js
[2026-09-30T04:10:09.121Z] GET /js/modelo.js
[2026-09-30T04:10:09.121Z] GET /js/vista.js
[2026-09-30T04:10:09.201Z] GET /api/incidentes
```

Figura 9. Reinicio del proceso: dotenv carga DATABASE_URL y GET /api/incidentes responde.